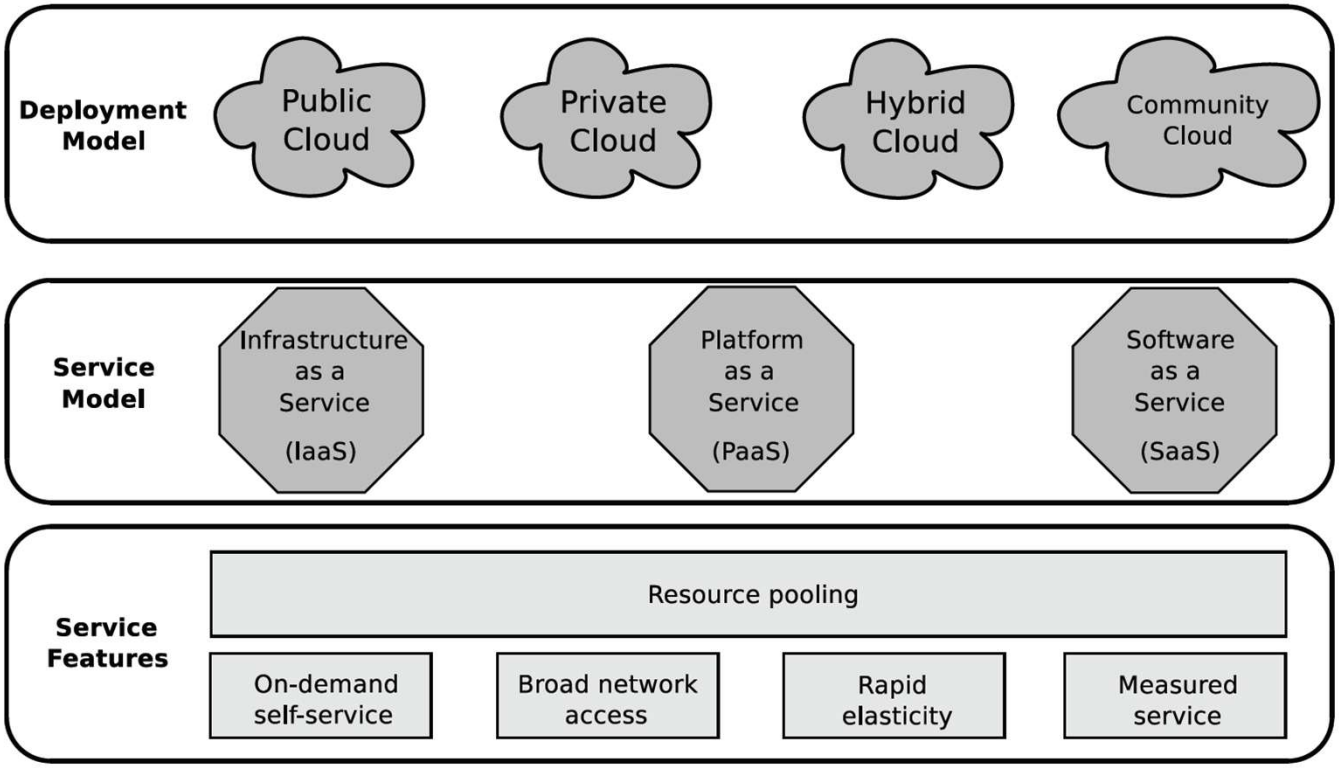
CLOUD COMPUTING

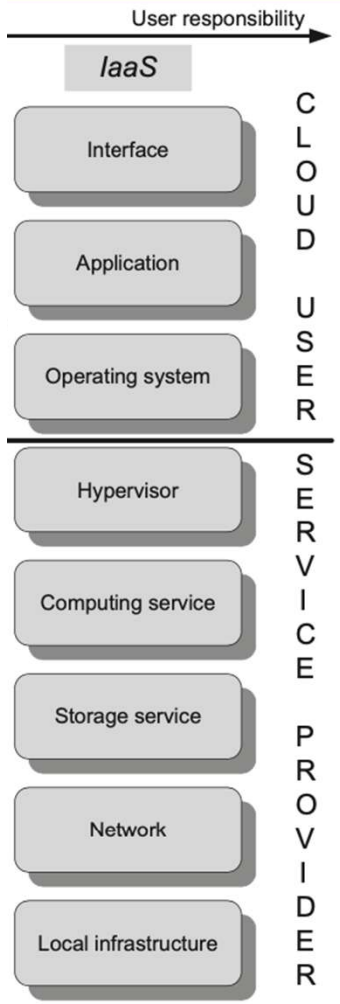
ENABLING TECNOLOGIES

NIST DEFINITION OF CLOUD COMPUTING



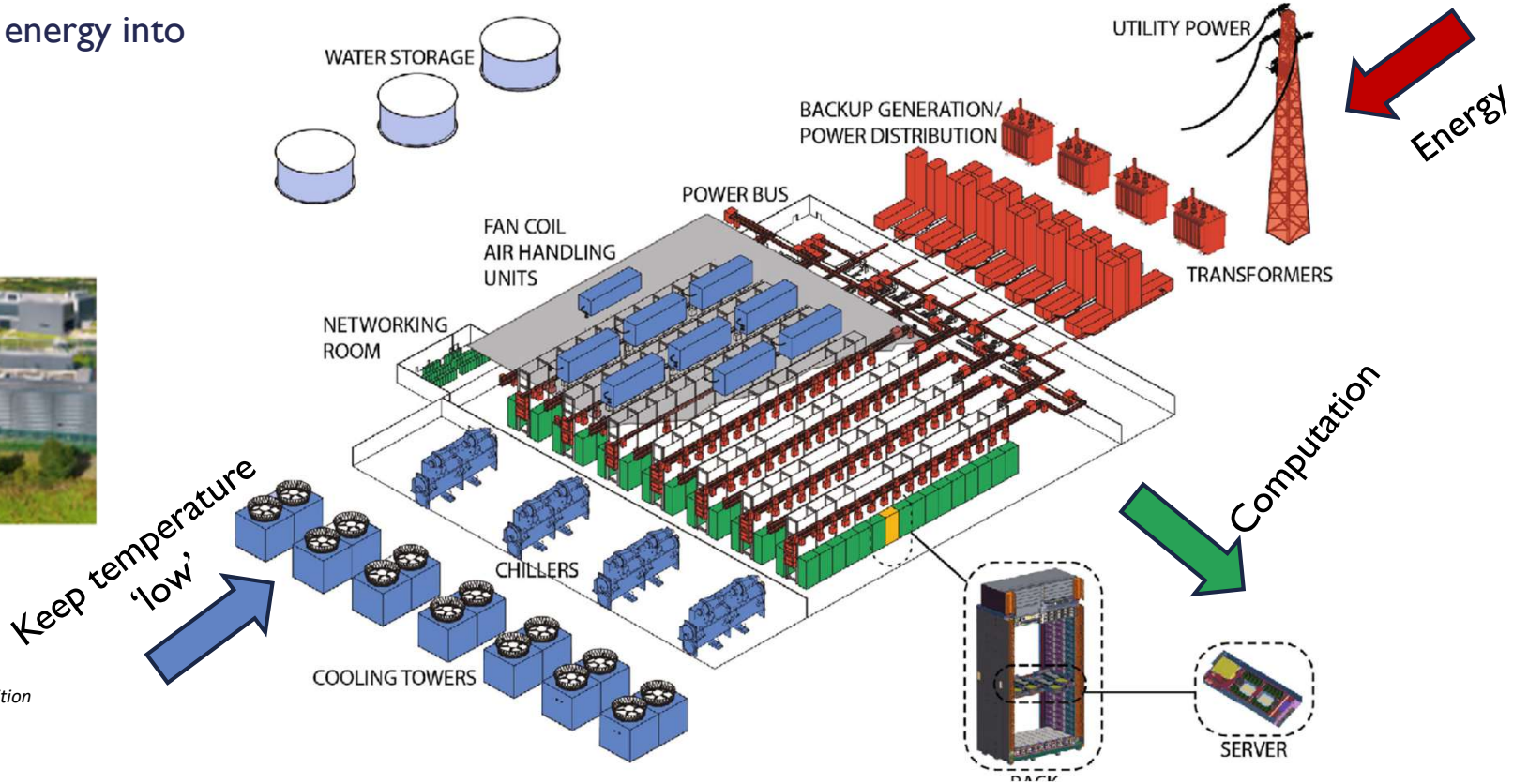
BEHIND IAAS

- IaaS are evolution of classic Data Center (DC)
- A DC is a controlled, secure environment ensuring continuous and reliable digital services. It includes
  - **Cooling systems:** air conditioning and centralized cooling to maintain optimal temperature.
  - **Power supply:** stable electrical power with backup systems.
  - **Fire protection:** sprinklers or gas-based systems to prevent equipment damage.
  - **Security:** personnel, access control, surveillance, and other protective measures.



DATA CENTER TECHNOLOGY

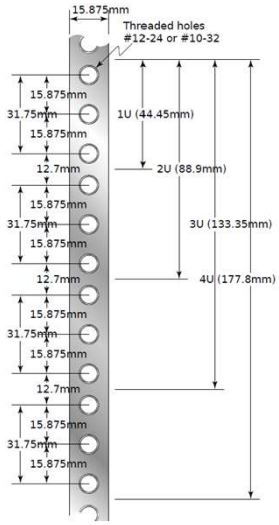
- A DC "transforms" energy into computation



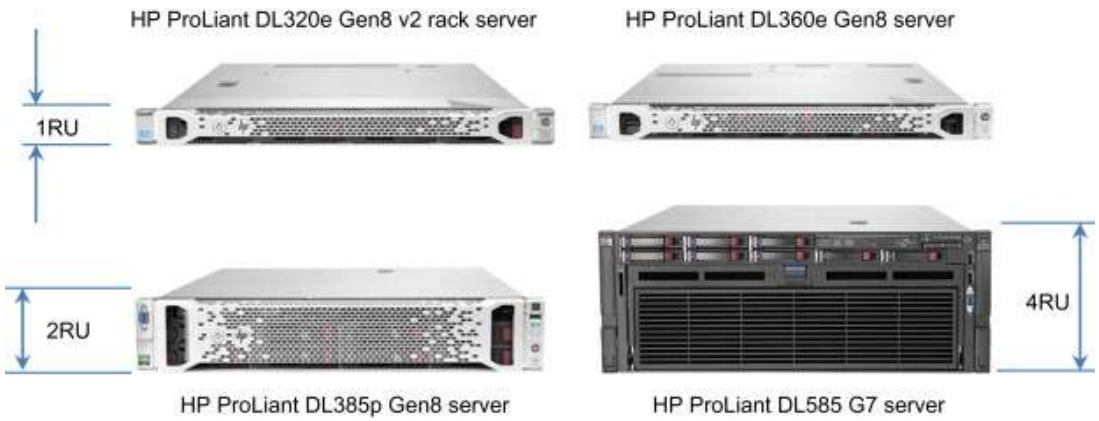
The Datacenter as a Computer
Designing Warehouse-Scale Machines Third Edition
Luiz André Barroso, Urs Hölzle, and Parthasarathy Ranganathan Google LLC
SYNTHESIS LECTURES ON COMPUTER ARCHITECTURE #46

COMPUTING UNITS

- The hardware building blocks of a DC are low-end servers, with rack enclosure (rack-mounted) or a blade enclosure
- Rack-mounted have all the components to run stand-alone (including power-supply), whereas blades need external supply and consequently need less space
- The size of a rackmount server is conventionally indicated in Rack Units (RU or U) (U=44.45mm)
- A sever space ranges from 1U to 4U



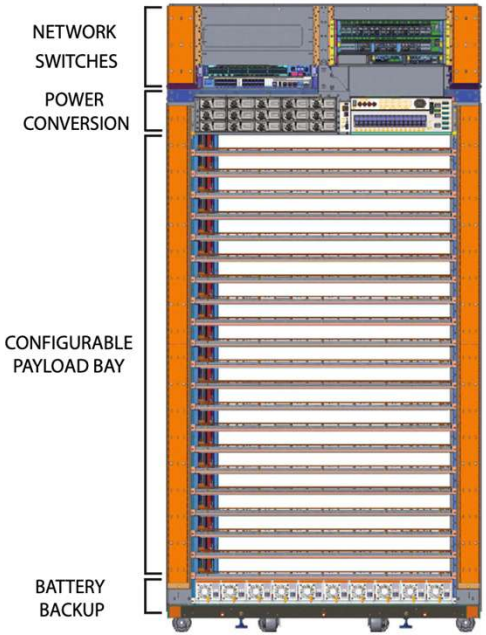
SOME EXAMPLE OF SERVER RACK



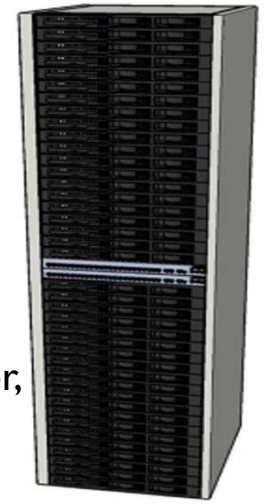
- Server grade hardware
  - hardware is designed for 24/7 operation.
- Mainboards usually contain > 2 CPUs per board and have larger caches.
- RAM sizes are usually very large in size (> 1TB).
- Network interface cards (NIC) are usually equipped with multiple ports, different port specifications and have high bandwidths(>10 GBit/s!).
- Storage capabilities are usually designed for fault tolerance and performance (e.g. RAID 6).

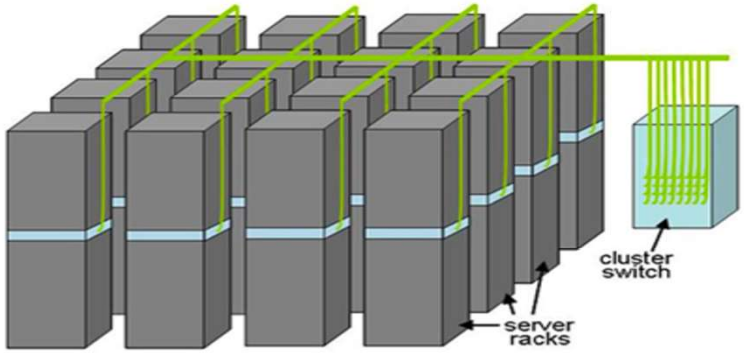
RACK OF SERVERS

- A standard rack consists of a payload bay to usually host 42 units (server, storage) and networking devices (switches, routers) known as ToR (Top o Rack) switch
- Communication occurs among servers inside the same rack, or with external endpoints



NETWORKING





- The traffic is not confined within servers of the same rack; rather, it can be directed to other racks (east–west traffic) or outside the DC (north–south traffic)
  - For example, Ref [*] shows how a single Facebook HTTP request generated 88 cache lookups (648 KB), 35 database lookups (25.6 KB), and 392 remote procedure calls (257 KB).
- To avoid communication bottleneck, the commutation topology is hierarchical (fat-tree of leaf-span)

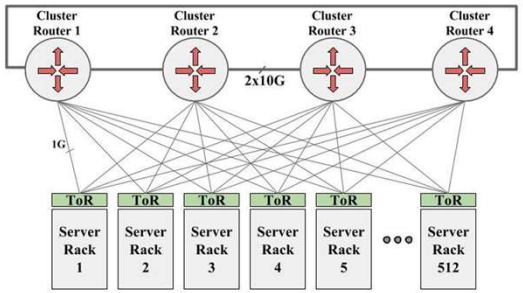
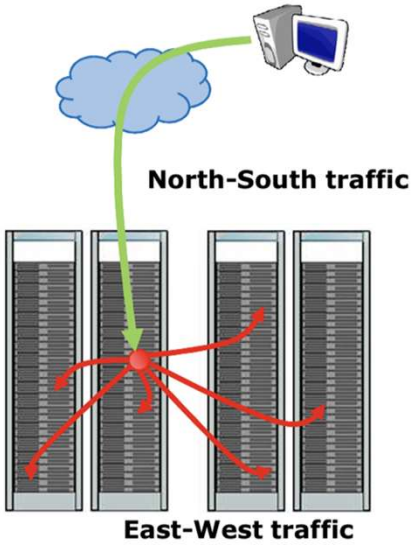


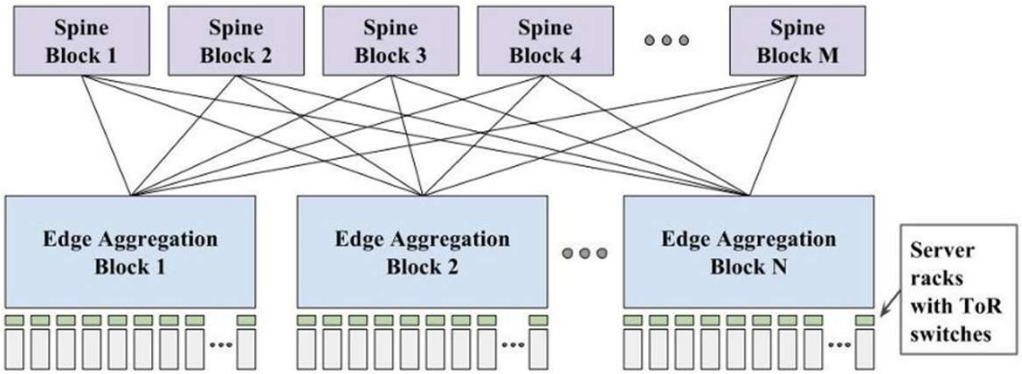
Figure 2: A traditional 2Tbps four-post cluster (2004). Top of Rack (ToR) switches serving 40 1G-connected servers were connected via 1G links to four 512 1G port Cluster Routers (CRs) connected with 10G sidelinks.

In addition, new communication protocols that allow parallel multi-path communications can be used instead of single path, TCP



(*) N. Farrington and A. Andreyev. Facebook'sdata center network architecture. In Proc. IEEE Optical Interconnects, May 2013

LEAD-SPINE TOPOLOGY



https://engineering.fb.com/2014/11/14/production-engineering/introducing-data-center-fabric-the-next-generation-facebook-data-center-network/

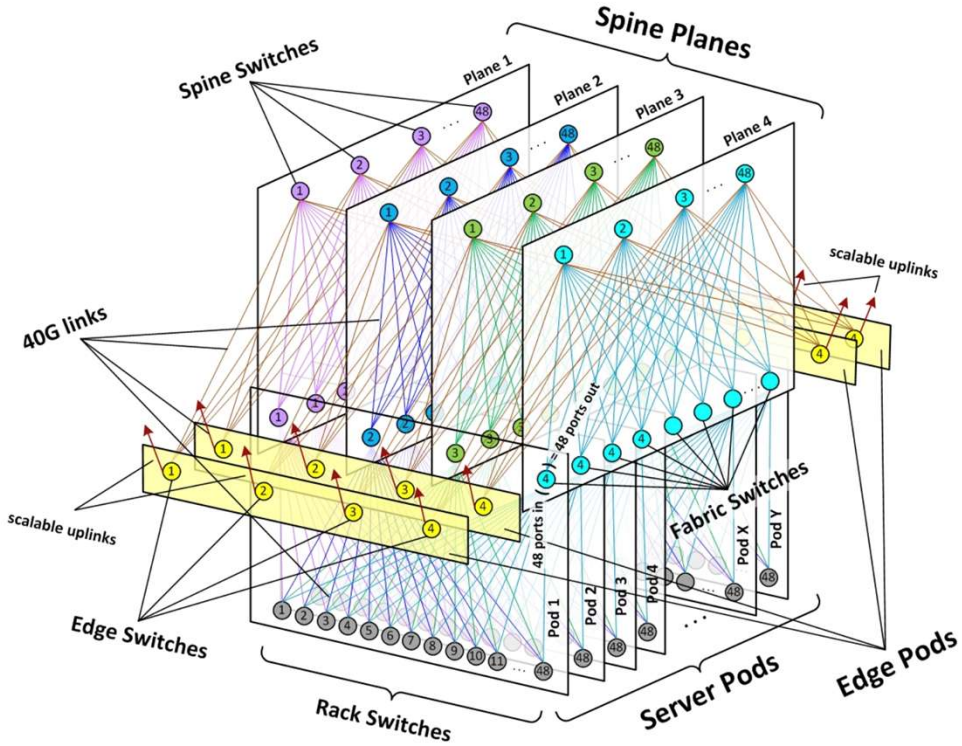


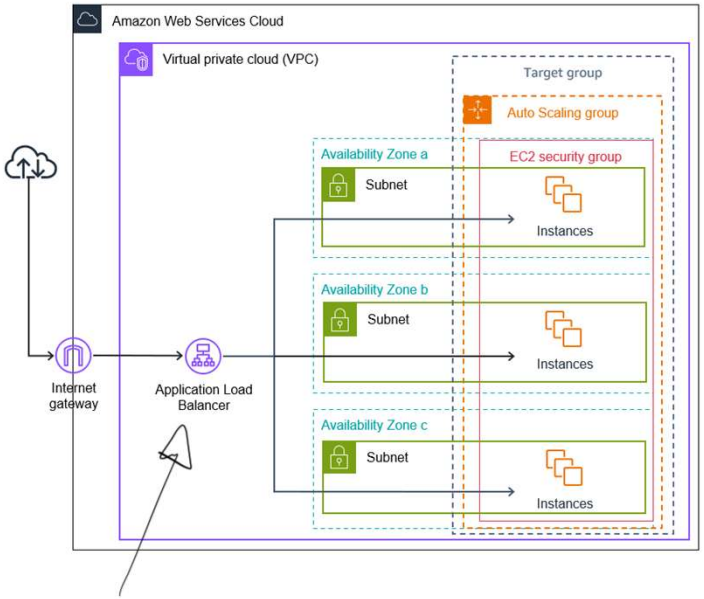
Figure 2: Schematic of Facebook data center fabric network topology

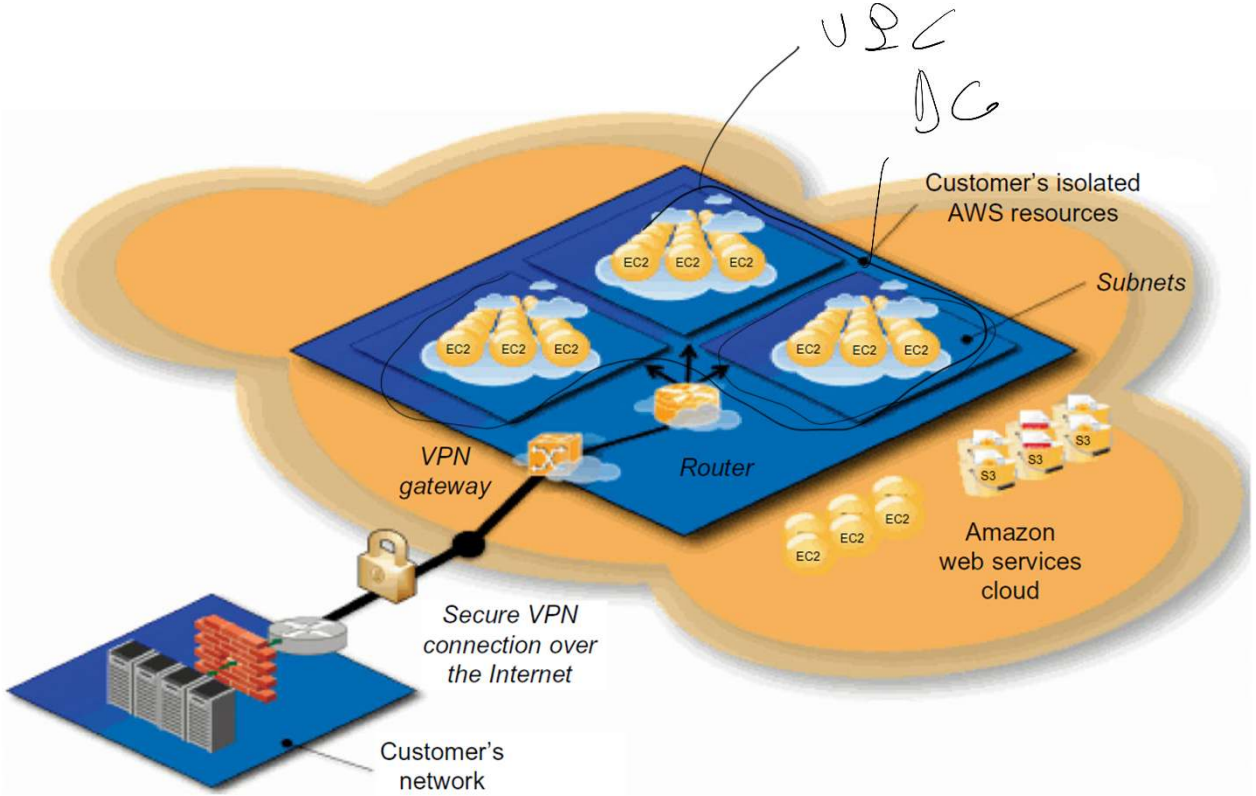
NETWORKING

- An Availability Zone (AZ) is a name used to denote single physical data center, or more often, a group of physically separate data centers located close to each other, sharing network infrastructure and low-latency connectivity.
- AZ further aggregated in regions (a region usually has 2–6 Azs)
- Latency is lower for users of that region
- To increase availability, user traffic can be distributed among different AZ of a region using an Application (level) Load Balancer (ALB)

VIRTUAL PRIVATE CLOUD

- A Virtual Private Cloud (VPC) is a set of subnets in different DCs





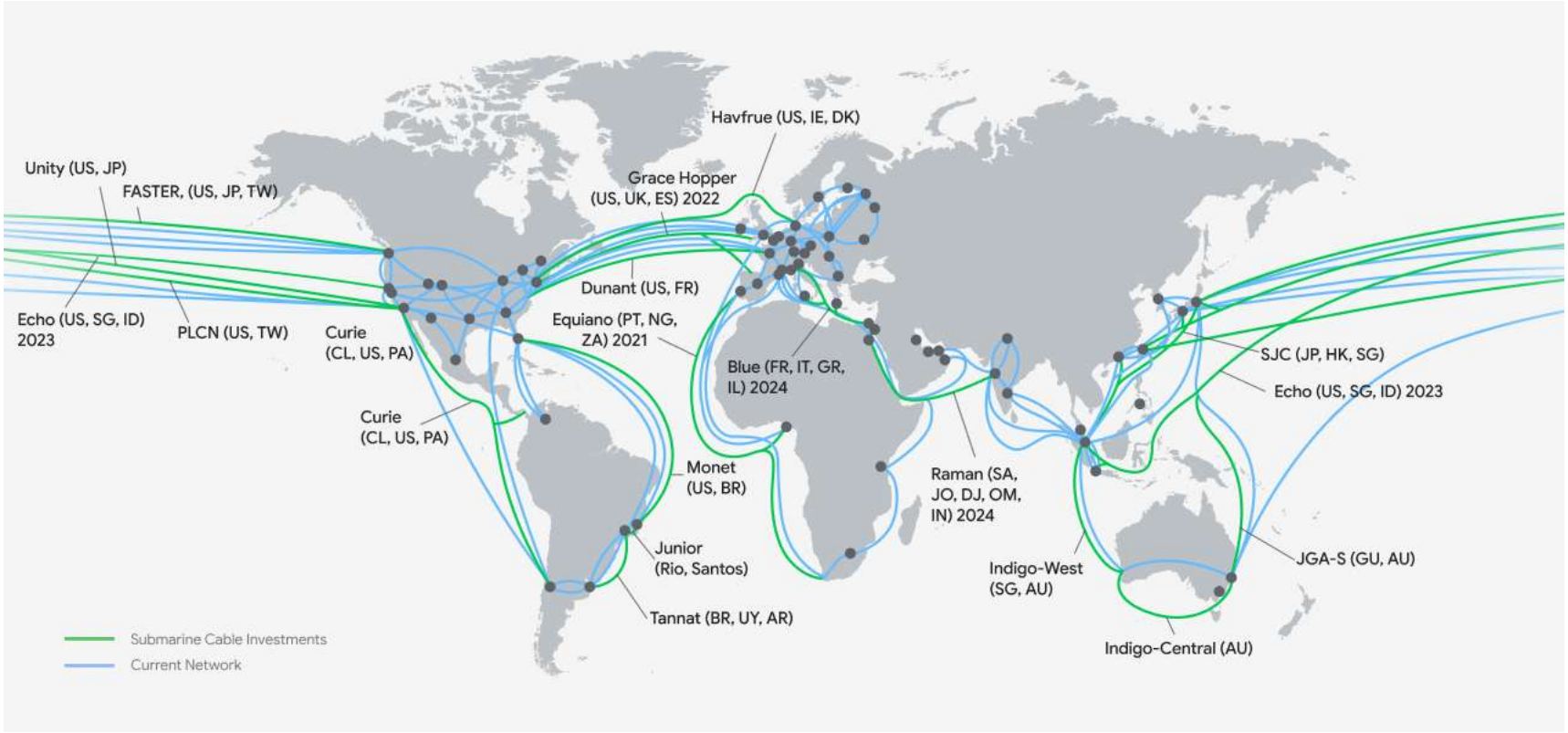
Credits: Distributed and Cloud Computing From Parallel Processing to the Internet of Things

EXAMPLE: GOOGLE CLOUD PLATFORM (GCP)

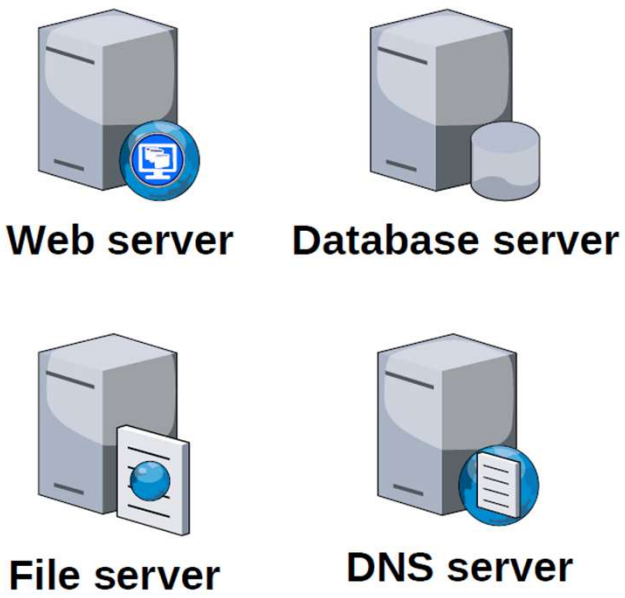
- Google Cloud's infrastructure follows a similar philosophy:
- The infrastructure is based in five major geographic locations: North America, South America, Europe, Asia, and Australia, connected via high-speed networks and subsea cables
- Locations are currently divided into 40 regions and regions into 121 zones

GCP LOCATIONS

GCP REGIONS


Havfrue (US, IE, DK)
Unity (US, JP)
FASTER, (US, JP, TW)
Grace Hopper
(US, UK, ES) 2022
Dunant (US, FR)
Echo (US, SG, ID)
2023
PLCN (US, TW)
Curie
(CL, US, PA)
Equiano (PT, NG,
ZA) 2021
Blue (FR, IT, GR,
IL) 2024
SJC (JP, HK, SG)
Echo (US, SG, ID) 2023
Curie
(CL, US, PA)
Monet
(US, BR)
Raman (SA,
JO, DJ, OM,
IN) 2024
Junior
(Rio, Santos)
Indigo-West
(SG, AU)
JGA-S (GU, AU)
Tannat (BR, UY, AR)
Indigo-Central (AU)
Submarine Cable Investments
Current Network

TYPE OF SERVERS



- Web server
  - Designed for the hosting of websites.
  - Supports web standards (HTTP/S) and frameworks (HTML/JavaScript).
- Database server
  - Specialized for data storage and retrieval (Relational/NoSQL).
  - Concurrent access of data by users.
- File server
  - Serves files or file systems to clients.
  - Supports standard protocols (FTP/S).
- DNS server
  - Specialized for the name resolution in computer networks.
  - Functionality by DNS-Software

SERVER VIRTUALIZATION

Bare metal sever

- One standalone physical server with one dedicated OS installed on the server
- Exclusive access to all hardware resources (CPU, RAM, storage, networks)
- Suitable for high performance, because of direct access to hardware capabilities
- **Drawbacks**
  - Management and maintenance is complex
  - Management and maintenance is very expensive
  - High cost for operation of servers
  - Dedicated usage of server leads to many ***idle times***

VM server

- Virtual server, which a 'slice' of the bare metal server
- It acts like a hw server, i.e. allows to run applications that usually require a dedicated server

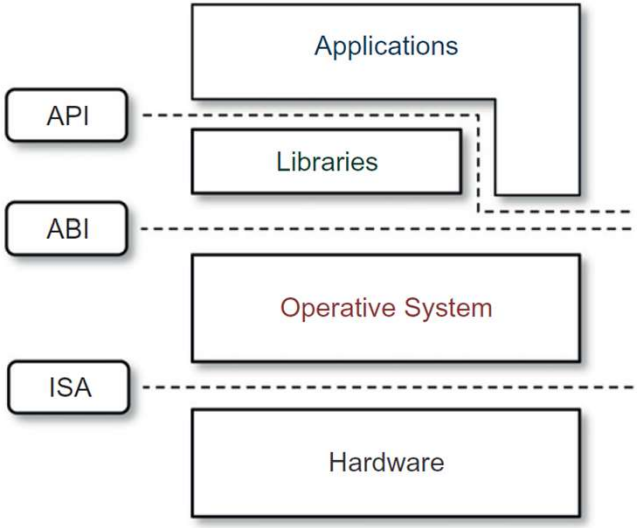
HW VIRTUALIZATION

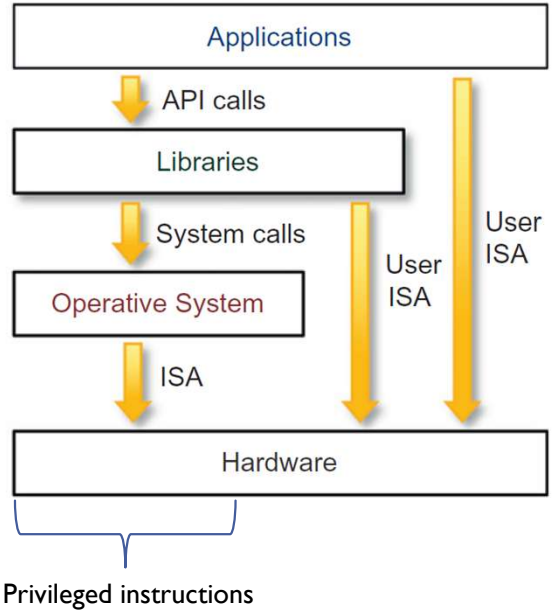
- Hardware virtualization involves virtualizing the hardware of a server (called **host**) by creating a Virtual Machine (VM) that provides the illusion of a physical machine (**guest**), using a special software called Virtual Machine Manager (VMM) or Hypervisor. VM old idea (1964 IBM's CP-40).
- Each virtual machine, is composed of a virtual CPU (**vCPU**), memory, I/O, and network devices
- This sounds familiar: exactly what an OS does to support multi-programming, right?
- Each process sees its own CPU, Memory, IO etc. so what is the difference with VMM?
- What if one just map a tenant to a user → tenant = user ???

HW VIRTUALIZATION

- A VM provides much higher **isolation** respect to multi-users solution:
  - In a multi-user system:
  - A user can see who else are using the server
  - Processes IDs are unique system-wide
  - It easier to compromise the system if a user is attacked ..
- But, an OS is different than a process

MACHINE REFERENCE MODEL



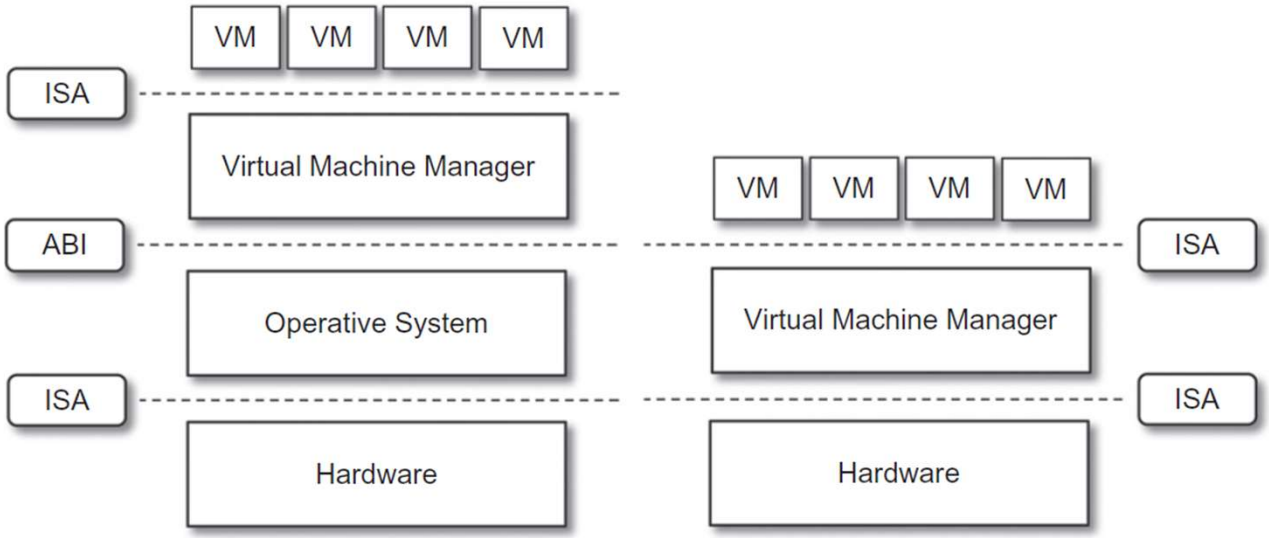



```python
import math

x = 16
y = math.sqrt(x)  # call to the math library API
print(y)          # output: 4.0
```

```c
#include <unistd.h>

int main() {
    const char *msg = "Hello, world!\n";
    write(1, msg, 14); // file descriptor 1 = stdout
    return 0;
}
```

```
MOV EAX, EBX
MOV [ECX], EAX
```

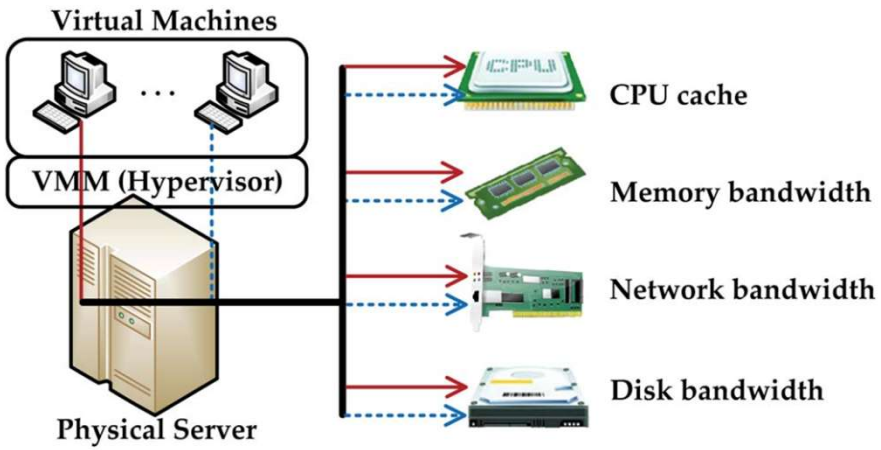
VIRTUAL MACHINE MANAGER



VMM AND PRIVILEGED INSTRUCTIONS

- Privileged instructions may vary the state of the physical CPU, or **pCPU** (like disabling interrupts).
- These instructions are dangerous if used maliciously
- .. should change the flag in the vCPU, not in the pCPU
- This is achieved by managing the state of the vCPU through the VMM either completely by sw (trap-and-emulate) or with hw assistance

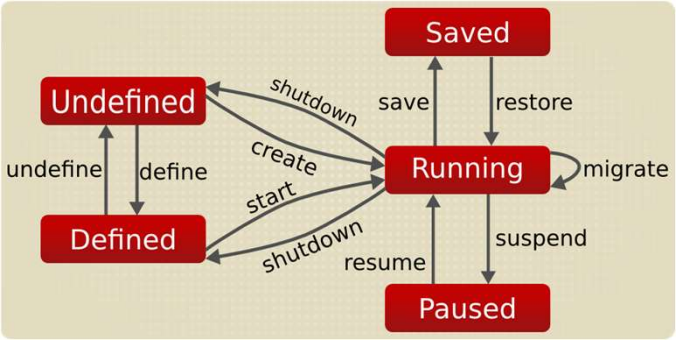
NOISY NEIGHBOURS



- Depending on the load on the other VMs, cache miss rate may for example increase, network bandwidth decrease, etc, ('noisy neighbours')
- In general, when the number of VMs increases behind the real pCPU capabilities, e.g. due to VM **overcommitment**, the vCPU clock runs at lower rate
- Wall clock correction

BENEFIT OF VM

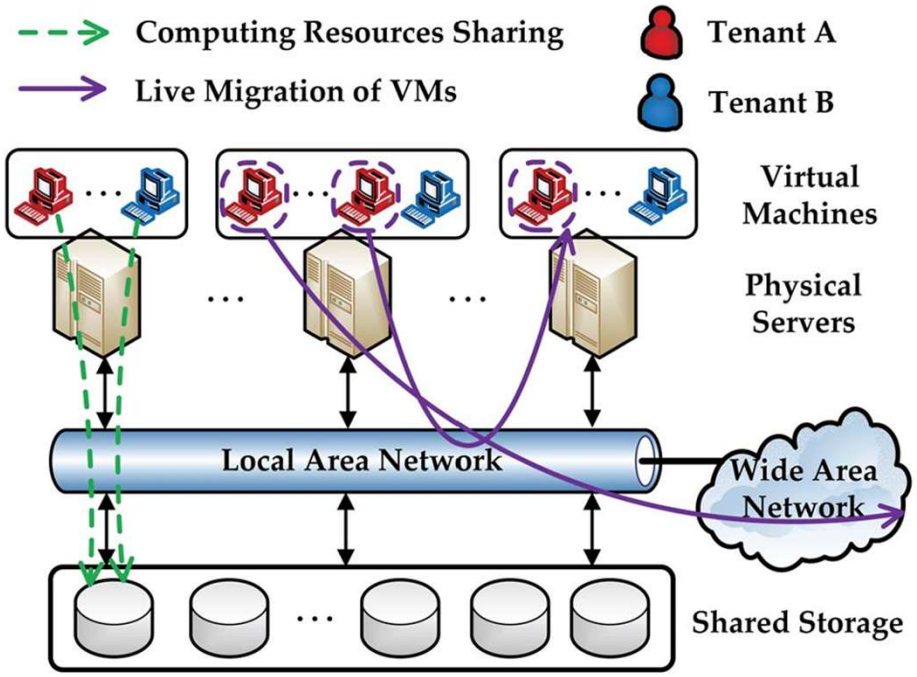
- High isolation among tenants
- Server consolidation
- VM management



BENEFIT OF VM (CONT.)

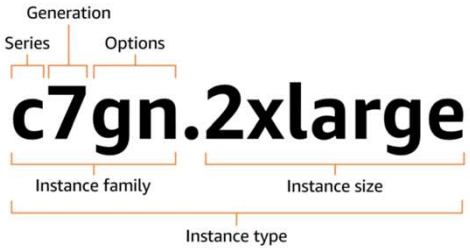
- **Templating** – create an OS + application VM, provide it to customers, use it to create multiple instances of that combination
- **Live migration** – move a running VM from one host to another
  - No interruption of user access

EXAMPLE: LIVE MIGRATION OF VM



EXAMPLE: EC2

- Amazon Web Service (AWS), provides a large options of pr VM, suitable for different purposes
- For example,
- t3.medium → 2 vCPU, 4 GB RAM
- m5.large → 2 vCPU, 8 GB RAM
- r5.xlarge → 4 vCPU, 32 GB RAM



Series	Options
• **C** – Compute optimized • **D** – Dense storage • **F** – FPGA • **G** – Graphics intensive • **Hpc** – High performance computing • **I** – Storage optimized • **Im** – Storage optimized (1 to 4 ratio of vCPU to memory) • **Is** – Storage optimized (1 to 6 ratio of vCPU to memory)	• **a** – AMD processors • **b200** – Accelerated by NVIDIA Blackwell GPUs • **g** – AWS Graviton processors • **i** – Intel processors • **m1ultra** – Apple M1 Ultra chip • **m2** – Apple M2 chip • **m2pro** – Apple M2 Pro chip • **b** – Block storage optimization • **d** – Instance store volumes
• **Inf** – AWS Inferentia • **M** – General purpose • **Mac** – macOS • **P** – GPU accelerated • **R** – Memory optimized • **T** – Burstable performance • **Trn** – AWS Trainium • **U** – High memory • **VT** – Video transcoding • **X** – Memory intensive • **Z** – High memory	• **e** – Extra storage (for storage optimized instance types), extra memory (for memory optimized instance types), or extra GPU memory (for accelerated computing instance types). • **flex** – Flex instance • **n** – Network and EBS optimized • **q** – Qualcomm inference accelerators • ***tb** – Amount of memory for high-memory instances (3 TiB to 32 TiB) • **z** – High CPU frequency

IMPLEMETATION OPTIONS

- **Type 0 hypervisors** - Hardware-based solutions that provide support for virtual machine creation by HW partition. They do not manage VM, e.g. do not allow migration, rather partition and HW allocation to OS
  - IBM LPARs and Oracle LDOMs are examples
  - Used on mainframe or high-end servers
- **Type 1 hypervisors** - Operating-system-like software built to provide virtualization
  - Including *VMware* ESX, *Citrix* XenServer, Nitro Hypervisor (by ASW plus custom HW cards)
- **Type 1 hypervisors** – Also includes general-purpose operating systems that provide standard functions as well as VMM functions (e.g. as kernel module)
  - Including Microsoft Windows Server with HyperV and RedHat Linux - KVM

IMPLEMETATION OPTIONS

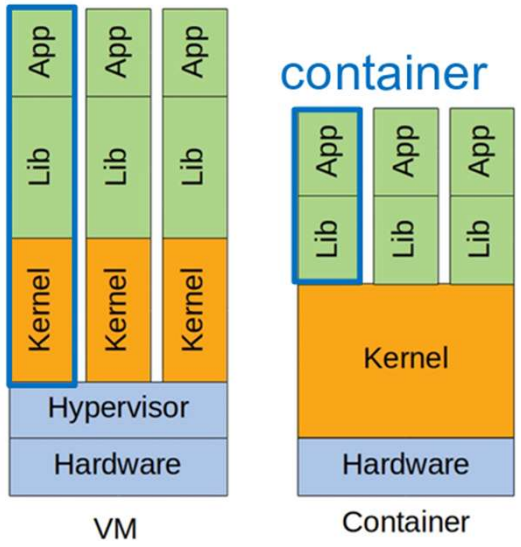
- **Type 2 hypervisors** - Applications that run on standard operating systems but provide VMM features to guest operating systems
  - **Including** VMware Workstation and Fusion, Parallels Desktop, and Oracle VirtualBox

IMPLEMENTATION OPTIONS

- Other variations include:
  - **Paravirtualization** - Technique in which the guest operating system is modified to work in cooperation with the VMM to optimize performance
  - **Emulators –** Allow applications written for one hardware environment to run on a very different hardware environment, such as a different type of CPU

IMPLEMENTATION OPTIONS

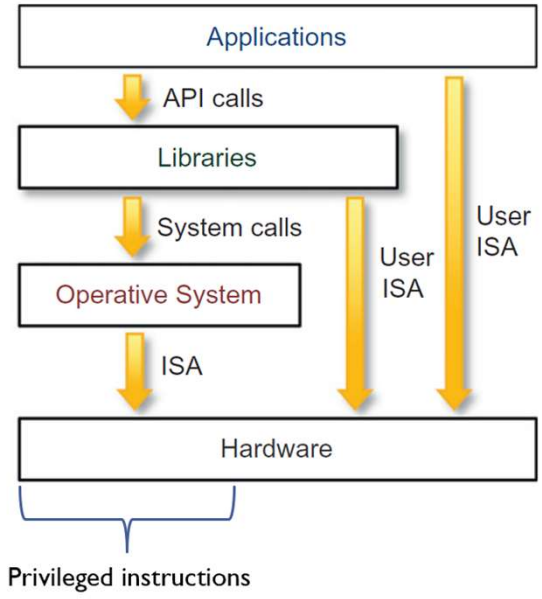
- **Application containment** - provides virtualization-like features by segregating applications from the operating system, making them more secure, manageable
  - Including Oracle Solaris Zones, BSD Jails, and IBM AIX WPARs
  - Linux Containers (LXC)
  - Docker containers



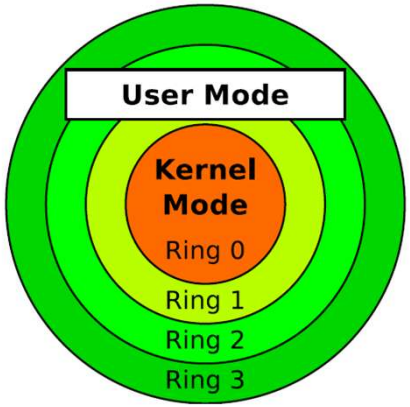
BUILDING BLOCKS OF A VMM

- Most VMMs implement a **virtual CPU** (vCPU) to represent state of CPU per guest as guest believes it to be
  - When guest context switched onto CPU by VMM, information from vCPU loaded and stored (much like processes in a OS, vCPU like PCB Process Control Block)
  - Most of the binary instructions are performed without any change by the physical CPU (pCPU)
- Several techniques

BASICS OF THE X86 ARCHITECTURE



- x86-compatible CPUs implement 4 privilege levels (or rings), indicated by a register called CPL (Current Privilege Level).
  The current state of the CPU is given by the value of the CPL.
- In ring 0 (kernel mode), the CPU can execute all machine instructions. The kernel runs in ring 0
- In ring 3 (user mode), the CPU cannot execute certain privileged operations; user applications must run only in ring 3.
- Modern, unmodified operating systems typically use only these two privilege levels (ring 0 and ring 3).
- If a user-mode process needs to perform a higher-privileged operation (e.g., access hardware), it does so by invoking a system call.
- This generates a controlled exception, which is intercepted in ring 0 and handled by the kernel.



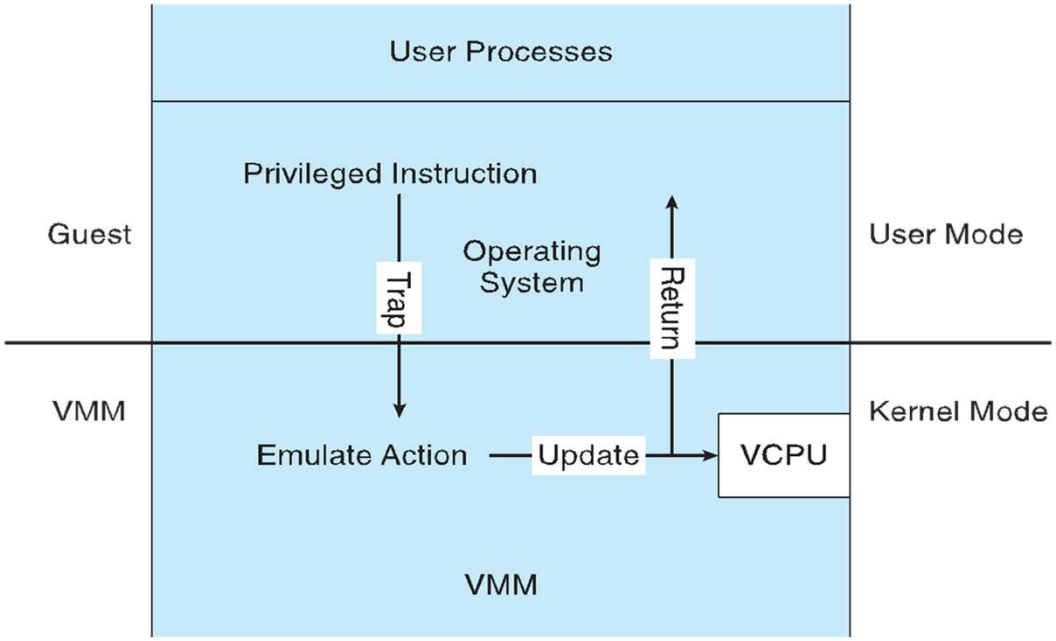
BUILDING BLOCK – TRAP AND EMULATE

- A VM also needs two modes – virtual user mode and virtual kernel mode
- In trap and emulate the OS runs in real user mode (de-privileged, runs when the CPU is in ring3)
- Actions in guest OS that usually cause switch to kernel mode must cause switch to a virtual kernel mode

TRAP-AND-EMULATE (CONT.)

- How does switch from virtual user mode to virtual kernel mode occur?
  - Attempting a privileged instruction in user mode causes an error → trap
  - VMM gains control, analyzes error, executes operation as attempted by guest
  - Returns control to guest in user mode
- User mode code in guest runs at same speed as if not a guest
- But kernel mode privilege mode code runs slower due to trap-and-emulate
  - Especially a problem when multiple guests running, each needing trap-and-emulate
- CPUs adding hardware support, mode CPU modes to improve virtualization performance

TRAP-AND-EMULATE VIRTUALIZATION IMPLEMENTATION

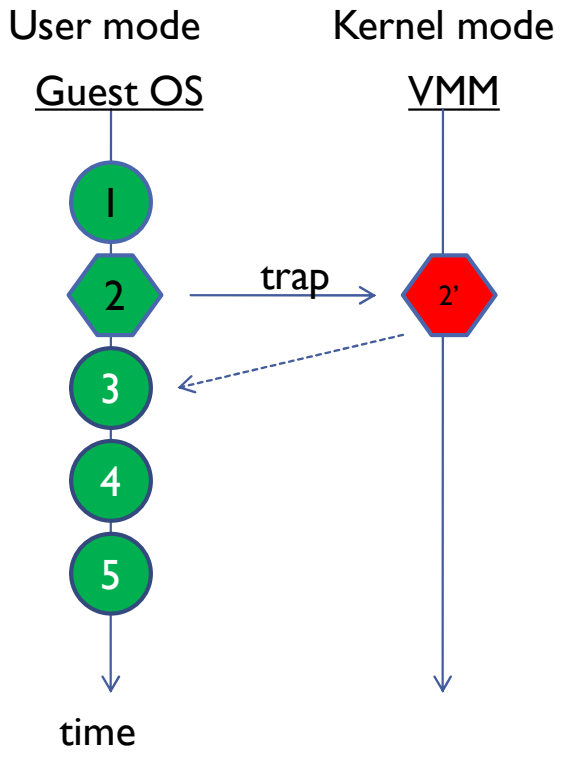


EXAMPLE

1. Not privileged instruction
2. CPU executes a privileged instruction of the Guest OS while being in user mode
3. CPU generates a trap
4. Control passes to the VMM that emulates the instruction
5. 2' is different of 2, but is produces the same effect
   - for example, CLI means pCPU.IF=0. becomes vCPU.IF=0)
6. Control returned to the guest OS

Privileged instruction
e.g.. CLI, Clear Interrupts

Not privileged instruction



green= User Mode red= Kernel Mode

POPEK AND GOLDBERG VIRTUALIZATION REQUIREMENTS

- Popek and Goldberg (1974) defined a set of conditions sufficient for a computer architecture to support system virtualization efficiently
- They classified machine instructions into 3 groups:
  - Privileged instructions: do not trap when the processor is in supervisor mode, but trap when in user mode
  - Sensitive instructions: change underlying resources (e.g., do I/O or change page tables) or observe information that indicates current privilege level (thus exposing that guest OS does not run on bare metal); for example, change the privileged state
  - Innocuous instructions: not sensitive
- Necessary condition: *For any conventional computer, a virtual machine monitor may be constructed if the set of sensitive instructions for that computer is a subset of the set of privileged instructions*

EXAMPLE: INSTRUCTION ACCESSING THE PRIVILEGE REGISTER (X86)

- **Innocuous instruction**
  - ADD EAX, 1 (adds 1 to the EAX register).
  - Does not touch privileged resources nor reveal anything about privilege level.
  - Always safe and does not require VMM intervention.
- **Sensitive instruction**
  - PUSHF / POPF (save or restore the processor flags, including the **Interrupt Flag** that controls interrupts).
  - This instruction can reveal the current privilege state or modify it.
  - If it does **not** trap in user mode → it becomes **problematic for virtualization**
- **Privileged instruction**
  - MOV CR3, EAX (changes the CR3 register, which contains the base of the page table).
  - If executed in **kernel mode** → works normally.
  - If executed in **user mode** → causes a **trap** → the VMM can intercept it.

TRAP-AND-EMULATE

- Some CPUs don't have clean separation between privileged and non privileged instructions (4004 was designed for a calculator)
- How to deal with these cases?

1. HW assisted
2. Binary-translation

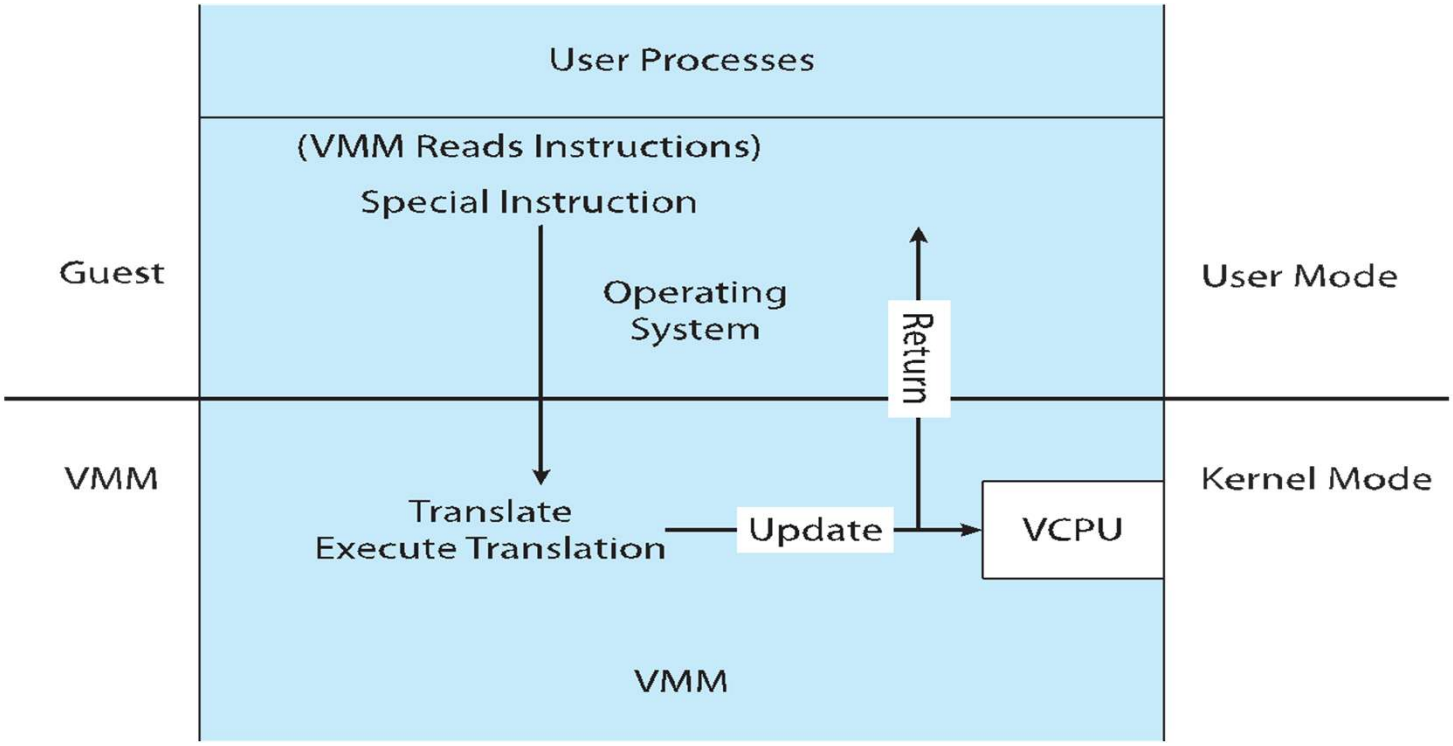
BINARY TRANSLATION

- Basics are simple, but implementation very complex
  1. If guest vCPU is in user mode, guest can run instructions natively
  2. If guest vCPU in kernel mode (guest believes it is in kernel mode)
     1. VMM examines every instruction guest is about to execute by reading a few instructions ahead of program counter
     2. Non-special-instructions run natively
     3. Special instructions translated into new set of instructions that perform equivalent task (for example changing the flags in the vCPU)

BINARY TRANSLATION

- Implemented by translation of code within VMM
- Code reads native instructions dynamically from guest, on demand, generates native binary code that executes in place of original code
- Performance of this method would be poor without optimizations
- Products like VMware use caching
  - Translate once, and when guest executes code containing special instruction cached translation used instead of translating again
  - Testing showed booting Windows XP as guest caused 950,000 translations, at 3 microseconds each, or 3 second (only 5 %) slowdown over native

BINARY TRANSLATION IMPLEMENTATION



BUILDING BLOCKS – HARDWARE ASSISTANCE

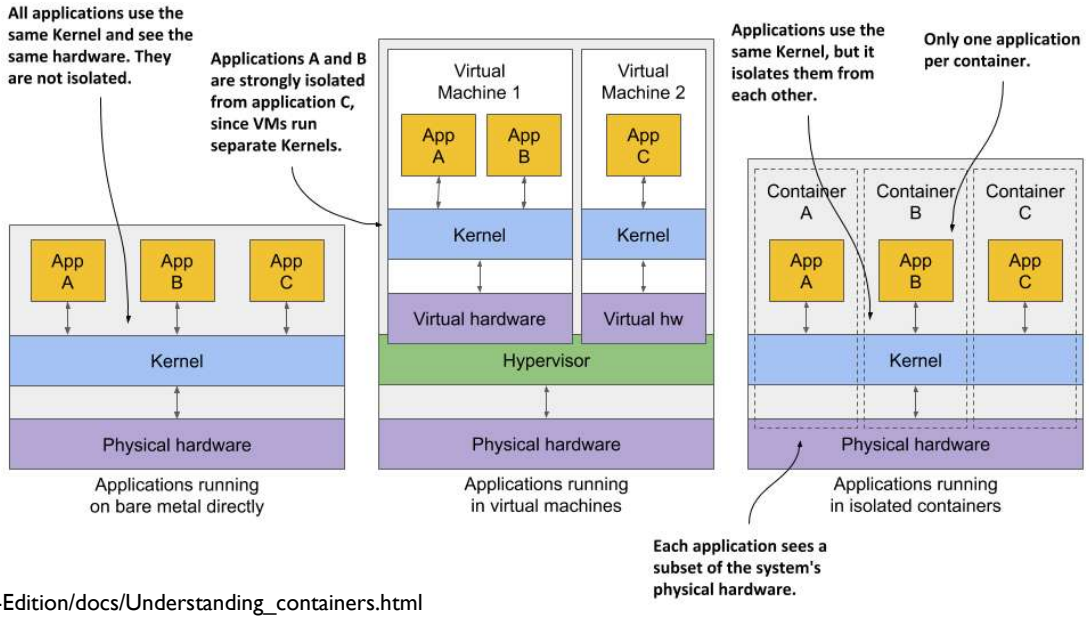
- Intel added new **VT-x** instructions in 2005 and AMD the **AMD-V** instructions in 2006
- CPUs with these instructions remove need for binary translation

EXAMPLE: WINDOWS SUBSYSTEM FOR LINUX (WSL)

- Provides access to bash shell and Linux tools directly inside Windows (10 or 11)
- Versions
  - WSL 1 → translates Linux system calls into Windows calls (lightweight but limited compatibility)
  - WSL 2 → uses a real Linux kernel in a lightweight VM → full compatibility + better performance (supports Docker)
- Advantages
  - Run Linux tools and libraries without dual boot or heavy VMs
  - Integration with Windows: file system, network, processes
  - Great for cross-platform development
  - Supports Docker Desktop, Kubernetes

CONTAINERS

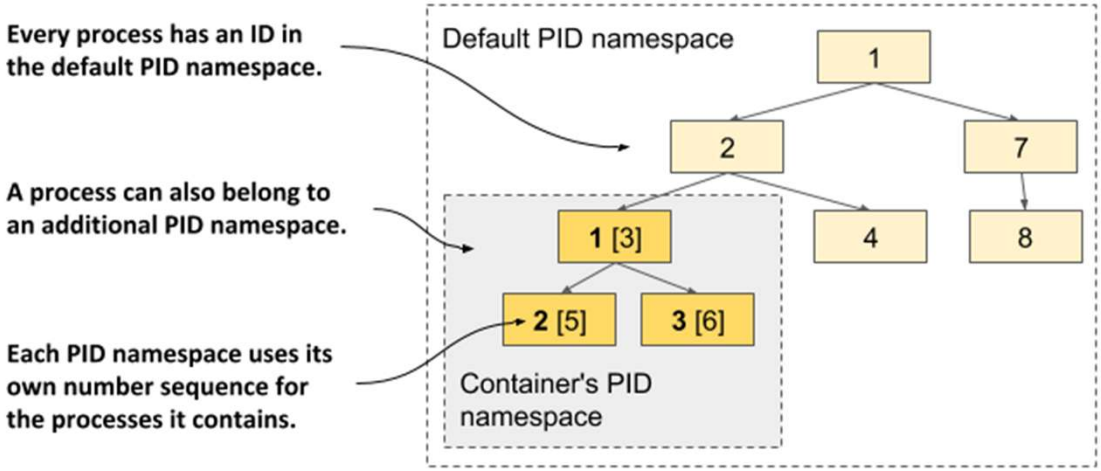
- A container is a group (tree) of processes with additional isolation compared to vanilla processes
- Historically, UNIX-style operating systems have used the term *jail* to mean a process that has limitations now replaced with container
- Containers implement OS-level virtualization



https://wangwei1237.github.io/Kubernetes-in-Action-Second-Edition/docs/Understanding_containers.html

NAMESPACE (LINUX-SPECIFIC)

- The kernel provides process isolation by creating separate namespaces for containers.
- Each container has its own set of namespaces
- For example, the PID namespace allows processes inside the container to have their own process IDs starting from 1



https://wangwei1237.github.io/Kubernetes-in-Action-Second-Edition/docs/Understanding_containers.html

ISOLATION

- **Process**
  - Each container, being a processes, has its own PID (from the host perspective), but also an ID with numerations starting from 1 (PID=1 is the init process of the container).
  - Processes created inside a container are independent of the host and other containers. For example, two processes in different containers can have the same 'internal' PID without conflict.
- **Filesystem**
  - Containers have their own root filesystem.
  - Without this mechanism isolation is just based on permissions and all processes see the same FS structure
  - Overlay/**Union** is an optimization mechanism that avoids duplications
- **User**
  - Containers can have their own user and group IDs, which can be mapped to host users differently.
- **Network**
  - Each container can have its own network interfaces, IP addresses, routing tables, and firewall rules.

NAMESPACES

- **MNT** isolates mount points
- **PID** isolates the PID space, so that each process only sees itself and its children (PID 1, 2, 3, …)
- **NETWORK** allows each container to have its dedicated network stack: its own private routing table, set of IP addresses, socket listing, firewall, and other network-related resources
- **USER** isolates user and group IDs, e.g., allowing a non-root user on the host to be mapped with the root user within the container
- **UTS** (Unix timesharing): provides dedicated host and domain names (the name of the host can be different in containers)
- **IPC** provides dedicated shared memory for IPC, e.g., different Posix message queues

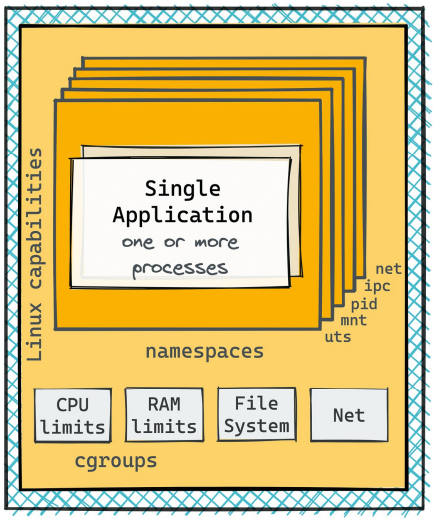
CONTROL GROUP (CGROUP)

- A cgroup (control group) is a group processes
- Each container has its own cgroup, and all the processes created from the container belongs to the group
- Kernel uses a cgroup to assign and monitor resources of (all the processes in) the container
- CPU time, system memory, network bandwidth

- While namespace defines what a container can see, cgroup determines how much of those resources can be used

CONTAINERIZATION

Container - a "box" for one app



typically has a dedicated address

e.g. 172.18.0.3

Credits: https://www.sahaj.ai/deep-dive-into-container-internals/

OS-LEVEL VIRTUALIZATION VS HYPERVISOR TYPE-1

- Minimal performance degradation: apps makes syscall directly, not via the VMM
- Short startup and shutdown times: from minutes per VM to seconds (or even less)
- Small footprint: hundreds of containers per server
- Containerized apps highly portable

- Less flexibility: cannot run on different OS, windows containers cannot run on linux
- Higher risk of vulnerability: vulnerability in the OS compromise the entire system; a single compromised container can compromise the OS or other containers

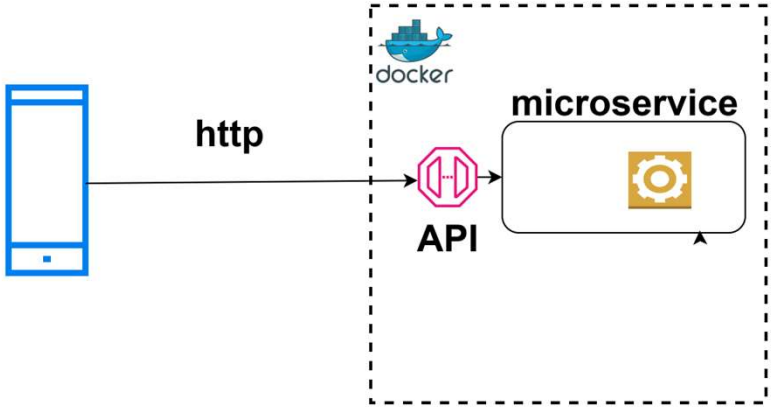
SOME PRODUCTS

- Docker: the most popular engine for application containers . Simplifies container management
- LXC: LinuX Containers: maintained by the mainline Linux kernel
- Podman: Supports OCI (Open Container Initiative for standard)

- Orchestrator is a sw for provisioning, deploy, monitor and dynamical control containers
- For example, docker swarm, Kubernetes (K8)

CONTAINERS AND MICROSERVICES

- Containers are the key technology for microservices
- A microservice is a small, independent, and self-contained service that performs a specific function within an application.
- Microservices communicate with each other or with clients through (web) APIs



CONTAINER-AS-A-SERVICE(CAAS)

- Container and container orchestrators are first-class Cloud services (CaaS)
- Allow to manage and deploy containerized applications
- Examples:
- Elastic Container Service (AWS)
- Azure Container Instance (MS)
- Cloud Run (GCP)

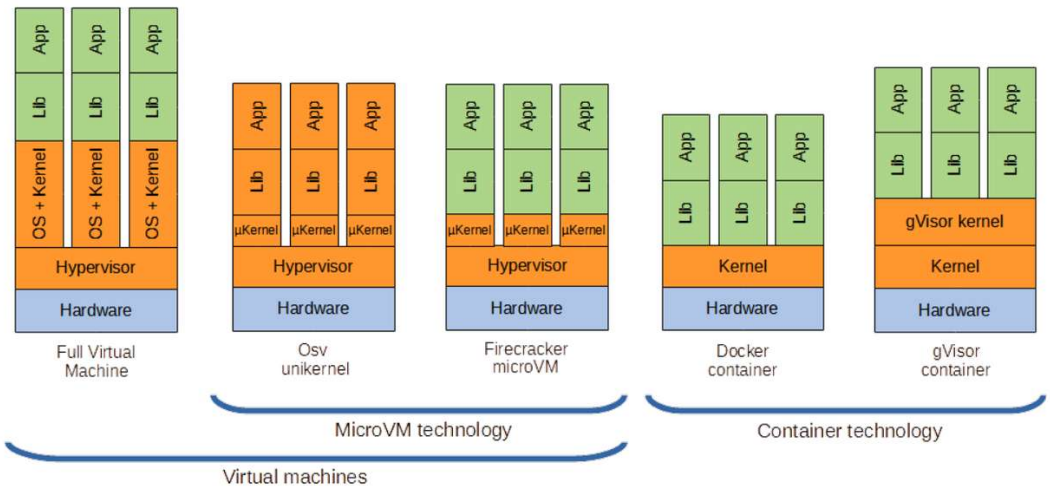
- Google GKE - Google Kubernetes Engine,
- Amazon EKS - Amazon Elastic Kubernetes Service,
- Microsoft AKS – Azure Kubernetes Service,
- IBM Cloud Kubernetes Service,
- Alibaba Cloud Container Service.

SERVLESS AND FUNCTIONS AS A SERVICE

- Serverless computing means running code without having to manage servers.
- Containers can be used to realize serverless computing.
- For example, Cloud Run allows a user to define their own Docker image, which is launched only when a request for the container arrives from a client.
- The service also manages scaling automatically.
- Because the running code can be seen as just a function call, this paradigm is called Function as a Service (FaaS)
- Lambda is a FaaS from AWS

MICROVM AND UNIKERNELS

- Micro-VM refers to VMM that are lightweight, i.e. they have small memory footprint and small boot time (some 100 ms)
- For example, Firecracker is VMM built by AWS to support the lambda service
- Unikernels are specialized OS where (the part of the used) kernel is included as a library



WHICH TECHNOLOGY IS BETTER?

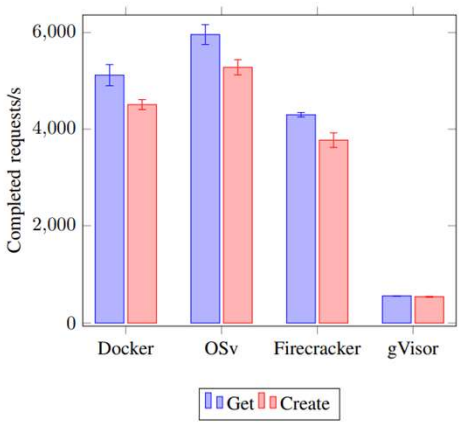


Fig. 3. Request processing performance in different virtualization solutions on **ARM**. *Higher is better.* The performance penalty of gVisor is even more pronounced on ARM systems. Unikernels still show a performance improvement over other guest approaches.

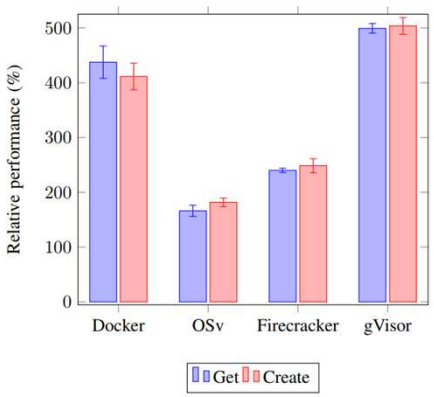


Fig. 5. Relative performance of multi-threaded vs single-threaded reques processing on ARM. *Higher is better.* While Docker and gVisor scal superlinearly, implying amortized overhead, OSv and Firecracker are not abl to fully utilise additional cores.

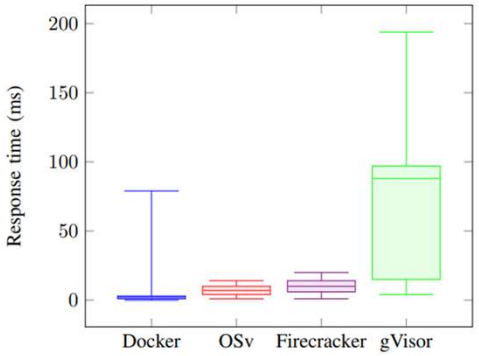


Fig. 6. Response latency variation of different virtualization solutions on ARM. *Lower is better.* While Docker has significantly lower average latency, its maximum latency varies significantly. Virtualization-based platforms have significantly more stable maximum response latency, even though their average latency is higher.

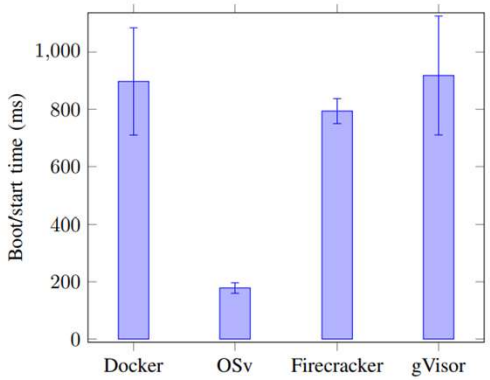


Fig. 9. Boot and start times of the service for the evaluated virtualization platforms on ARM. *Lower is better.* OSv boots in just 200ms, while Firecracker performs slightly better than Docker.

T. Goethals, M. Sebrechts, M. Al-Naday, B. Volckaert and F. De Turck, "A Functional and Performance Benchmark of Lightweight Virtualization Platforms for Edge Computing," *2022 IEEE International Conference on Edge Computing and Communications (EDGE)*,